

Primer trabajo de computación paralela

CUDA (9%)

En este trabajo se pretende hacer una evaluación básica de **CUDA** en **Google Colab** (usando su **versión gratuita**, **NO** la versión Premium).

Elabore un programa en CUDA (**use lenguaje C**, **NO** use C++ ni otro lenguaje compatible con CUDA) que haga lo siguiente:

Se trabajará con el conjunto de datos de Kaggle Forest Cover Type (en Kaggle, *uciml/forest-cover-type-dataset*): 581.012 parcelas de bosque en Colorado, cada una descrita por 54 variables (elevación, pendiente, orientación, distancias a fuentes de agua y a carreteras, índices de sombra y 44 indicadores binarios de área silvestre y tipo de suelo) y **etiquetas con uno de siete tipos de cobertura forestal** → **Variable a predecir**.

Fuente y licencia del conjunto de datos: El conjunto se descarga de Kaggle: <https://www.kaggle.com/datasets/uciml/forest-cover-type-dataset>

Es un espejo del conjunto Covertype del repositorio de aprendizaje automático de la Universidad de California en Irvine: <https://archive.ics.uci.edu/dataset/31/covertime>

Cada parcela corresponde a una celda de 30 por 30 metros del Bosque Nacional Roosevelt, en Colorado.

De ahí se toman dos conjuntos (muestras) de manera aleatoria de 20 mil parcelas **cada una**, sin que ninguna parcela quede en las dos:

- **Conjunto de referencia:** son las 20 mil parcelas cuyo tipo de cobertura **sí** se conoce, y son las que se emplearán para clasificar al conjunto de consulta.
- **Conjunto de consulta:** son las otras 20 mil parcelas que hay que clasificar. Su tipo de cobertura se elimina de esta muestra de datos (ya que será la variable a predecir).

Para **clasificar una parcela del conjunto de consulta** se hace lo siguiente:

1. Calcular la distancia entre esa parcela y cada una de las 20 mil parcelas del conjunto de referencia. La distancia es la Euclidiana: se restan las 54 variables, se eleva al cuadrado cada diferencia y se suman los 54 resultados. **NO** saque la raíz cuadrada, porque para saber cuál distancia es menor no hace falta, y así se ahorra esta operación en cada una de las 400 millones de parejas que hay que evaluar.
2. Conservar las **k**-parcelas de referencia de menor distancia, junto con sus etiquetas correspondientes al **tipo de cobertura forestal**. El valor de **k** se define al inicio del programa.
3. Asignarle a la parcela del conjunto de consulta la etiqueta que más se repita entre esas **k** parcelas. Si hay empate, gana la etiqueta de menor valor.

Lo anterior se hace para cada una de las 20 mil parcelas del conjunto de consulta.

Preparación de los datos

El procesamiento de datos lo puede hacer en Python.

1. Descargar el conjunto de datos.
2. Tomar 20 mil parcelas como conjunto de referencia y 20 mil como conjunto de consulta.

3. Normalizar cada una de las 54 variables para cada parcela en los dos conjuntos: restarle su media y dividirla por su desviación estándar. La media y la desviación se calculan únicamente con el conjunto de referencia (y estos valores se usan en los dos conjuntos para llevar a cabo la normalización). Si la desviación de alguna variable es cero (porque esa variable toma el mismo valor en todas las parcelas de la referencia), cámbiela por 1 antes de dividir (para evitar dividir por cero). **Nota:** Si no se normaliza, la variable **elevación** (que se mide en miles de metros) domina por completo la distancia y las otras 53 variables dejan de influir...

4. Escribir tres archivos planos (.bin): variables (del conjunto) de referencia, etiquetas (del conjunto) de referencia, variables (del conjunto) de consulta.

Su programa debe hacer lo siguiente:

1. Leer los tres archivos .bin preparados anteriormente.
2. Implementar la versión **secuencial** del algoritmo en C, **sin** usar CUDA. Esta versión es la implementación secuencial contra la cual se comparan las versiones paralelas, con respecto al tiempo.
3. Copiar las variables de referencia y de consulta al device, copiar las etiquetas.
4. Invocar el kernel que se encarga de clasificar las parcelas del conjunto de consulta.
5. Copiar las 20 mil etiquetas resultantes desde el device hacia el host.
6. Imprimir los tiempos obtenidos, tal como se indica más adelante.

Reparto del trabajo entre los hilos

El kernel se invoca con un número de bloques y un número de hilos por bloque que usted decide (ver más abajo). Cada hilo se encarga de clasificar un grupo de parcelas del conjunto de consulta. Ejemplo: si hay 1000 hilos, a cada hilo le corresponden 20 parcelas.

Nota: Dentro del kernel imprima el número del bloque, el número del hilo y el rango de parcelas que le corresponde, para poder verificar que el reparto es el que se pidió. Haga esa comprobación con pocos bloques y pocos hilos: con veinte mil hilos imprimiendo, la salida es ilegible. Procure que los mensajes sean claros, de forma que durante las pruebas se observe qué le toca a cada hilo.

Presente tablas de resultados (tiempos de ejecución reportados por el programa), ensayando con diferentes valores para **k**, número de bloques y número de hilos. Ensaye con **8** combinaciones distintas de número de bloques e hilos. Por ejemplo, un bloque con un solo hilo (acá se tiene un kernel escalar, es decir, ejecución secuencial en la GPU), 100 bloques cada uno de a 10 hilos, etc.

Nota: Ensaye al menos con **4** valores de **k = 3, 15, 60**.

De cada ejecución anote: el tiempo de la versión secuencial, el tiempo de la transferencia de ida, el tiempo del kernel, el tiempo de la transferencia de vuelta, el tiempo total. Note que hasta acá son 24 pruebas (**8 x 3 = 24**) más la secuencial en la CPU. Presente conclusiones. Por ejemplo: cuál fue la configuración óptima, cuánto afecta la transferencia de datos hacia y desde el kernel, cómo cambia el tiempo al crecer **k** en la versión secuencial y en la paralela, se justifica el uso de paralelismo para este problema, etc.

En resumen, el trabajo es tomar 20 mil parcelas y clasificar a cada una (obtener su tipo de cobertura forestal) de acuerdo con otras 20 mil parcelas que ya están clasificadas usando diferentes combinaciones de bloques, hilos por bloque y valores de k.

Entrega

Un cuaderno de Google Colab con el código, las tablas impresas y las conclusiones.

Notas:

- Para entregar por email a fjmoreno@unal.edu.co con copia al monitor **Julian Salazar** jusalazard@unal.edu.co, el **miércoles 7 de octubre hasta las 11 am.**
- No se reciben trabajos en hora posterior. No se reciben versiones “mejoradas”.
- Grupos máximo de tres personas.
- Los trabajos deben ser independientes entre los grupos. Trabajos copiados así sea en un solo fragmento se califican con 0 (cero) en su totalidad para todos los integrantes.
- Cualquier duda consultarla personalmente o por email con el profesor o con el monitor.
- El monitor les puede ayudar con aspectos básicos del problema (por ejemplo, ayudarle a investigar o a solucionar algún aspecto específico), pero su función no es hacerle la práctica **ni está autorizado para cambiar las condiciones del trabajo.**
- Si hay errores en el enunciado por favor informarme lo más pronto posible para corregirlos.
- Si trabaja con **otras** herramientas o lenguajes, el trabajo se le califica con cero así funcione.

Atentamente,

Francisco Moreno, septiembre 23, 2025.

Enunciado y planteamiento del problema: Julian Ricardo Salazar Duarte

Revisado y modificado por: Francisco Javier Moreno Arboleda

Trabajo **certificado** por el **Caramel Center** (anagrama de Carmen Electra).

